



PROPUESTA DE SERVICIO

My Story Marker

Novelas personalizadas para regalar, escritas por un harness de agentes que se puede verificar

CLIENTE Cuentalia Regalos, S.L.

OCASIÓN Catálogo de Navidad 2026: la novela-regalo personalizada

FECHA 25 de septiembre de 2026

PRESENTA [NOMBRE_ESTUDIANTE]



Portada real de la novela de ejemplo (10 capítulos, PDF exportado por el sistema)

Un regalo único que hoy no se puede comprar hecho

Quién compra y para qué

1

Hijos y nietos

jubilaciones, 70 cumpleaños, homenajes a un padre o una abuela

2

Parejas

aniversarios, bodas, «cómo nos conocimos»

3

Padres y padrinos

un cuento en el que el niño es el protagonista (cumpleaños, comunión)

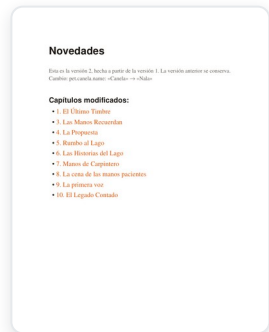
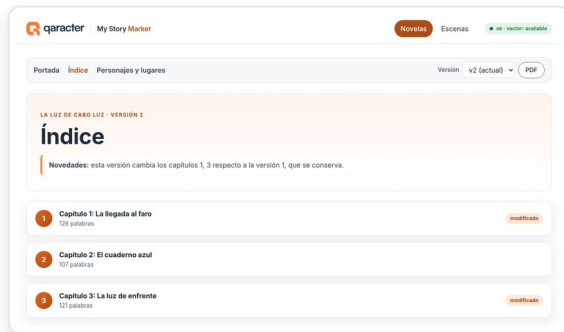
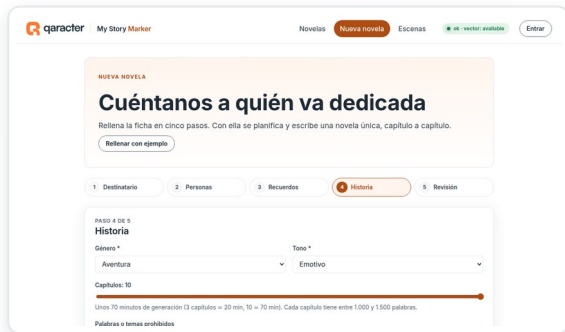
Por qué las alternativas actuales no funcionan

Alternativa	Por qué falla
Escritor por encargo	semanas o meses y precio de artesanía; un regalo tiene fecha
Libro plantilla	solo cambia el nombre: el destinatario no se reconoce
Chat con un LLM	mezcla nombres, olvida recuerdos, contradice fechas, ignora vetos; sin PDF ni correcciones

Lo que pide Cuentalia: 10 capítulos en horas, en los que el destinatario se reconozca, sin nada vetado y corregibles.

Criterio de calidad del sistema (D11): personalización y calidad narrativa pesan igual. Meter los datos a martillazos no cuenta como éxito.

Configurar, leer y corregir una novela



1 Configurar

Asistente de 5 pasos: destinatario, personas, recuerdos, historia (género, tono, vetos) y dedicatoria. El brief se valida con JSON Schema y reglas de coherencia antes de gastar un token.

2 Leer

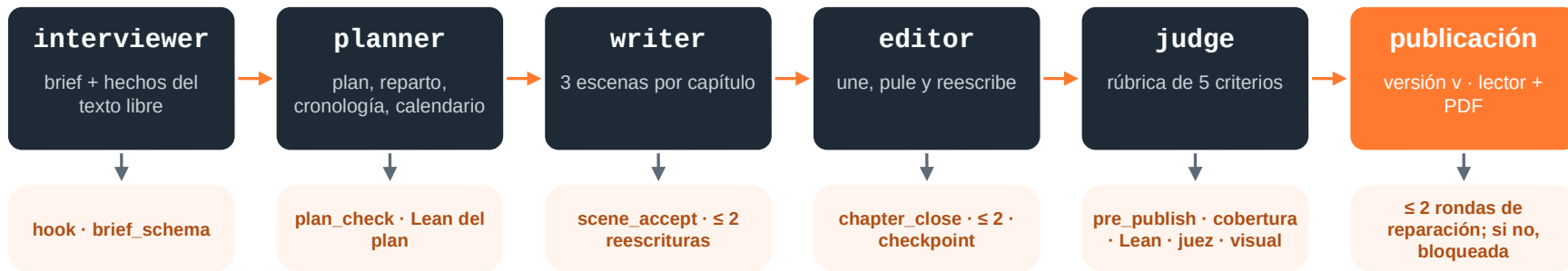
Lector web con portada y dedicatoria, índice, capítulos y fichas de personajes con «Aparece en». El PDF exportado tiene la misma estructura y enlaces internos.

3 Corregir

«La perra se llama Nala»: se pide desde el lector, la CLI o la API. Solo se regeneran los capítulos que usan ese hecho, en una versión nueva; la anterior se conserva.

Capturas reales: asistente «Nueva novela», validador visual del lector (Playwright MCP) y página «Novedades» del PDF de la versión 2.

Cinco roles con un solo trabajo, orquestados por código



⚡ Cada fallo vuelve, con su evidencia y un presupuesto de reintentos, al rol que puede arreglarlo. Agotado el presupuesto, la versión se bloquea: nunca se publica.

Orquestador · `app/novel/pipeline.py`

- Decide qué documentos ve cada rol y valida su salida con JSON Schema.
- Escribe en la story bible en nombre del rol: la tabla de permisos se aplica en código, no en un prompt.
- Ningún modelo tiene herramientas de escritura; todos corren en Claude Haiku 4.5.

Story bible

SQLite autoritativa: brief, hechos y su uso por escena, cronología, versiones, checkpoints y audit log.

Langfuse

Una sesión por novela; spans role:* y tool:*; tokens, coste, latencia, scores y prompts versionados.

Contexto mínimo y una sola fuente de verdad

1 Gestión del contexto

Cada rol recibe solo Documents: su trozo del plan, la checklist de hechos asignados, los resúmenes de capítulos previos y las últimas 250 palabras. Datos, nunca instrucciones; muy por debajo del límite de 100k tokens.

2 Story bible (SQLite)

17 tablas. Cada hecho del brief es una fila y fact_usage lo liga a (versión, capítulo, escena): así un cambio sabe qué regenerar. Texto y checkpoint en una transacción: se reanuda sin duplicar.

3 Tools

6 tools de solo lectura con schema: list_novels, list_versions, get_chapter, get_chapter_summary, query_story_bible, download_novel. Cada llamada es un span tool:<nombre>; también las expone un servidor MCP.

4 Hooks de Claude Code

PreToolUse policy_guard (bloquea escribir claves o borrar en la BD) y PostToolUse validate_chapter: el mismo código que el pipeline. Editar un capítulo a mano no se salta la validación. 14 casos probados.

Por qué este diseño

Orquestador determinista

frente a un agente con herramientas: permisos en código, reintento por paso y traza por rol.

Hechos estructurados

frente a RAG sobre la prosa: hechos exactos, saber dónde se usa cada uno y entrada directa a Lean.

Calendario en Python

frente a confiar en el LLM: los días de la semana no convergían entre reparaciones.

Haiku 4.5 en todo

coste y latencia de 10 capítulos; la calidad la vigila el juez bloqueante.

Cuatro tipos de validador, en cuatro puntos



Si falla: vuelve al rol productor con la evidencia (escena → writer, capítulo → editor, novela → capítulos citados). Cada resultado se guarda en validator_result y llega a Langfuse como score.

Resultados por brief, y la mejora medida tras el tuning

Brief	Antes	Después	Qué decidió (antes → después)	USD
b2-infantil	bloqueada	publicada	brief_coverage ✗ (frase literal) → todos ✓	0,97
b3-injection	bloqueada	publicada	brief_coverage ✗ · inyección ➤ → todos ✓ · inyección ➤ no seguida	1,02
b4-temporal	parada (plan)	bloqueada	error en el plan → judge_chapter / judge_novel ✗ (edad)	1,93
b5-contradiction	rechazado	rechazado	brief_schema ✗ → brief_schema ✗ (6 años + romance)	—

b4 bloqueada es el resultado esperado: su trampa de edad no tiene lectura coherente. b5 se rechaza antes de generar. Umbrales del juez intactos.

Qué cambió en el tuning 1

- brief_coverage: palabras de contenido normalizadas, no la frase literal del recuerdo.
- Planner con time_marker por capítulo; juez bloquea solo por un defecto «alta» concreto.
- Reparación solo de los capítulos citados; MAX_REPAIR_ROUNDS 1 → 2 (TLC re-ejecutado).

EVALS · 3 BRIEFS GENERABLES

0/3 → 2/3

novelas publicadas antes → después del tuning 1

10 CAPÍTULOS · TUNING 2

bloqueada → publicada

Calendario calculado en Python y calendar_consistency en chapter_close. Resultado: 10 capítulos, 10.645 palabras, 0 rondas de reparación, 3,10 USD.

Lean prueba la historia; TLC prueba el harness

Un fallo detectado por Lean (caso L04)

b4-temporal, sin el prechequeo del plan: la mascota muere en 2005 y lleva los anillos en la boda de 2008. lean_chronology falla en noAfterExit, señalando evento, fecha y capítulo.

Validador	¿Lo vio?
lean_chronology	Sí · noAfterExit
judge_novel	Sí · continuidad 2/5
judge_chapter	No · aprobó el capítulo
6 validadores programáticos	No

4 invariantes: temporalOrder · agesCoherent · noBilocation · noAfterExit. Corre en pre_publish (lake build) y sobre el plan antes de escribir (~0,12 USD frente a ~1,3 USD de una novela).

Propiedades verificadas con TLC

Propiedad	Garantiza
NoUnvalidatedPublish	no se publica nada que no pasó todos los validadores
ResumeNoDupNoLoss	reanudar tras un crash no duplica ni pierde capítulos
PreviousVersionKept	la versión publicada anterior no cambia
RetriesBounded	reintentos acotados, también tras un crash
EveryGenerationEnds	liveness: acaba publicada o en error

5.492.531

estados distintos, sin errores

N = 5 capítulos · 3 escenas · reintentos 2/2 · reparación 2 · profundidad 127 · 8 min 17 s. Antes, 4 contraejemplos (CE1–CE4) cambiaron el diseño.

Una traza real en Langfuse

Trace

b240be3a421522f38659648c978c5f2

latency 9m 42s

cost \$0.433949

tokens 155,212

session

quadrant:forbidden_words 1.00

1.00

1.00

1.00

judge:calidad_narrativa 4.00

4.00

4.00

+21

role:planner

2026-09-25 08:14:59.149

latency 2m 00s

Preview

Attributes

Scores

Log View

Formatted Raw

User

Diseña el plan completo de la novela: exactamente 1 capítulos numerados 1.1., cada uno con exactamente 3 escenas, cuyas word_budget sumen entre 1100 y 1350 por capítulo. Usa en facts_used las claves del documento bible/facts.txt: cada hecho con mandatory=true debe estar en al menos una escena. Al menos un evento de cronología por escena. Fechas en formato YYYY-MM-DD. Cada capítulo lleva time_marker (cuándo ocurre su acción respecto al anterior) y avanza en el tiempo salvo que marques flashback=true; ningún capítulo repite el núcleo de otro.

Assistant

title

La puerta abierta

synopsis

Tomás Arriaga, maestro rural durante treinta años, llega al final de su carrera en 2026. En la última clase que imparte en la escuela de Valdelosa, cierra la puerta de ese aula que lo recibió como joven maestro en 1992 con una maleta de cartón y una estufa rota. Treinta años enseñando con paciencia, generosidad y humor a generaciones de niños de montaña. Pero esta no es una historia de despedidas: es la aventura de descubrir que el final de una vida es el comienzo de otra. En el taller de carpintería donde ha buscado refugio durante años, Tomás y Canela reflexionan sobre lo hecho. Los muebles que construyó, la cuna de madera para su hijo Daniel, las manos que tallaron madera son las mismas que escribieron en pizarras. Cuando su familia lo encuentra en el taller, Tomás comprende que la jubilación no es un cierre, sino una puerta abierta hacia nuevas aventuras.

Captura real de Langfuse: spans por fase, capítulo, rol, tool y validador. 9 min 42 s · 0,43 USD · 155.212 tokens.

Scores (brief de inyección)

```
free_text_injection
```

flagged x2 (prescan + modelo)

forbidden_words_scene

9/9 · 1,0

prose_repetition

min 0,8

judge_chapter

3/3 · min 0,8

```
judge_novel . lean
```

aprobada

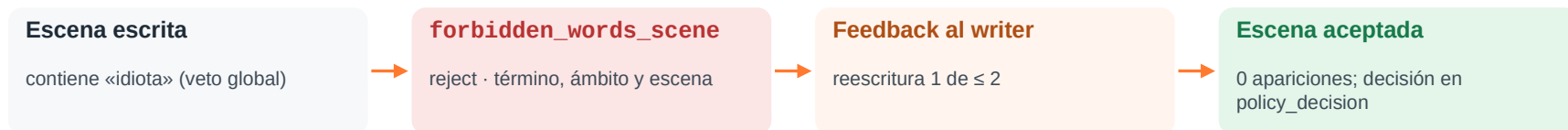
Prompts versionados usados

```
editor v4 . judge_chapter v2 . judge_novel v2 .
planner v4 . writer v4
```

Tokens: 4022 de entrada · 167.449 de salida · 141.307 de caché. Datos de `llm_call` (evals/results/after/b3-injection.json), la misma instrumentación que envía spans y scores a Langfuse.

Reglas que se cumplen aunque el modelo no quiera

Palabras prohibidas: detección y reescritura



Un solo normalizador (NFKD, sin tildes, plurales): detecta t0nt0 · tontooo · t.o.n.t.o · cabrones → cabrón; sin falsos positivos en «tontería» o «ridículo». Vetos en 3 niveles: global, del cliente, léxico.

1 Datos personales

- Los datos del destinatario viven solo en la story bible; cada novela tiene dueño y otro usuario recibe 404 (bcrypt + JWT).
- Claves fuera del repo; el hook bloquea escribirlas. Evals con destinatarios ficticios.

2 Injection

- Prescan determinista antes del modelo: 17/17 variantes. El extractor solo devuelve hechos y marca la sospecha.
- Ningún rol recibe el texto libre crudo: b3 marcó 5 patrones y se publicó sin seguirlos.

3 Audit log

- policy_decision, validator_result y llm_call: cada decisión, validador y llamada con prompt, tokens y coste.
- Nada se borra: una versión nueva nunca sobrescribe la anterior.

Coste medido y propuesta económica

3,10

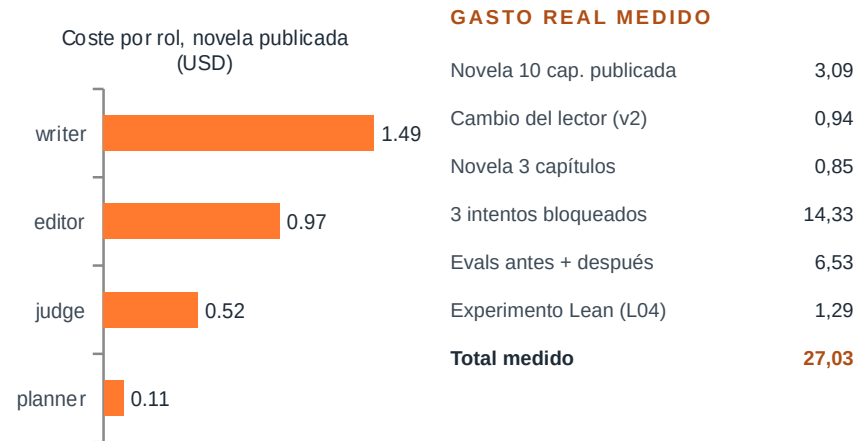
USD de modelo, novela de 10 capítulos publicada

0,94

USD por un cambio del lector (9 capítulos)

69 min

de generación, sin intervención



Novela publicada: 0,20–0,49 USD por capítulo, 55 llamadas, 472.436 tokens de salida. Techo por bucles acotados: el peor intento costó 6,05 USD y no se cobra (solo se factura lo publicado). Haiku 4.5: 1 USD/M tokens de entrada, 5 USD/M de salida.

PROPUESTA ECONÓMICA

Piloto de implantación (8 semanas)	28.800 €
integración con la tienda y login	8.000 €
lector y PDF con marca blanca	6.400 €
QA editorial: 50 novelas y calibración del juez	7.200 €
RGPD: enmascarado en trazas, borrado, DPA	4.800 €
formación y puesta en marcha	2.400 €
Plataforma (hosting, Langfuse, soporte)	600 €/mes
Por novela publicada (incluye 1 cambio)	12 €
Cambios adicionales	2 €

Ejemplo con PVP de 39 €: 12 € a Qaracter, 27 € de margen bruto para Cuentalia; la cuota se cubre con 23 novelas al mes. Precios sin IVA; 1 USD ≈ 0,90 € (supuesto).

Coste unitario, proyecto, volumen y sensibilidad

COSTE UNITARIO POR NOVELA

Concepto	€
Tokens medidos en Langfuse (3,10 USD)	2,79
Contingencia de reintentos (+30 %)	0,84
1 cambio del lector incluido (0,94 USD)	0,85
Infraestructura (150 €/mes ÷ 500 novelas)	0,30
Coste total	4,77
Precio a Cuentalia · margen operativo Qaracter	12 · 7,23 (60 %)

ESCENARIOS DE VOLUMEN (€/MES)

Novelas/mes	100	500	2.000
Ventas Cuentalia (PVP 39 €)	3.900	19.500	78.000
Margen Cuentalia	2.100	12.900	53.400
Margen Qaracter	1.203	4.214	15.504

PROYECTO DE DESARROLLO

Fase	Horas	€
Diseño (specs, arquitectura, TLA+)	60	4.800
Desarrollo (pipeline, lector, integración)	180	14.400
Validación (evals, Lean, QA editorial)	90	7.200
Despliegue (RGPD, puesta en marcha)	30	2.400
Total a 80 €/h	360	28.800 €

SENSIBILIDAD (POR NOVELA)

Escenario	Coste	Margen Qaracter
Base (1 cambio)	4,77 €	7,23 €
Tokens +50 %	7,01 €	4,99 €
4 revisiones (3 extra a 2 €)	7,31 €	10,69 €
6 revisiones sin cobrar	9,00 €	3,00 €
Tokens +50 % y 6 sin cobrar	13,35 €	-1,35 €

Con un precio fijo por novela, la subida de tokens la absorbe Qaracter; cobrar los cambios adicionales protege el margen. Tope recomendado: 3 revisiones incluidas.

«La perra se llama Nala»: solo cambia lo afectado

versión 1 · «Canela»

Un coche se detuvo fuera. Lucía apareció por la puerta del taller, viéndolo de pie junto a **Canela**.

—Aquí estás. Imaginé que vendrías aquí —dijo, sonriendo.

Él asintió sin palabras.

—¿Bien? —preguntó Lucía, acercándose.

—Sí. Solo necesitaba estar aquí un momento.

Ella apoyó una mano en su brazo, entendiendo.

—Anda, que tengo hambre. Te haré la comida que te gusta.

versión 2 · «Nala»

Un coche se detuvo fuera. Lucía apareció por la puerta del taller, viéndolo de pie junto a **Nala**.

—Aquí estás. Imaginé que vendrías aquí —dijo, sonriendo.

Él asintió sin palabras.

—¿Bien? —preguntó Lucía, acercándose.

—Sí. Solo necesitaba estar aquí un momento.

Ella apoyó una mano en su brazo, entendiendo.

—Anda, que tengo hambre. Te haré la comida que te gusta.

1

Petición en el lector: `pet.canela.name` → «Nala»

2

`fact_usage` dice qué capítulos usan el hecho: 1 y 3–10

3

Se regeneran solo esos, en la versión 2; el 2 se copia

4

Mismos validadores; se exporta el PDF con «Novedades»

44 → 0

«Canela» en v1 →
v2

0 → 45

«Nala» en v1 → v2

v1

intacta

~27 min

~0,94 USD

Misma página en [ejemplos/novela-ejemplo.pdf \(v1\)](#) y [ejemplos/novela-ejemplo-v2-cambio-nala.pdf \(v2\)](#). El índice de la v2 marca cada capítulo «(modificado)».

Lo que aún no está demostrado y cómo lo cerramos

Riesgo	Mitigación
Días sin fecha al lado («aquel lunes») y duraciones no se validan	extender calendar_consistency a la prosa
Lean solo prueba lo que el planner registra en la cronología	evento «departure» obligatorio si el brief dice que alguien se fue
Un brief contradictorio bloquea bien pero caro (b4: timeout de 45 min)	prechequeo del plan más estricto; parar antes de escribir
Fichas de personajes sin versionar (solo el nombre)	versionar reparto por versión
Datos personales reales en prompts y trazas	enmascarado en Langfuse, borrado a petición y DPA (piloto)
Juez aún no calibrado con revisión humana	QA editorial de 50 novelas con la misma rúbrica

Siguientes pasos

- 1** **Semanas 1–2**
integración y RGPD
- 2** **Semanas 3–5**
marca blanca y 50 novelas de QA
- 3** **Semanas 6–7**
calibración del juez y validadores nuevos
- 4** **Semana 8**
lanzamiento en el catálogo de Navidad



Gracias

Separamos la verdad de la prosa: lo que es cierto sobre la historia vive en una base de datos y lo comprueban validadores, Lean y TLA+. Solo se publica lo que pasa todo.

EMPRESA

Qaracter

CONTACTO

[NOMBRE_ESTUDIANTE]

WEB

qaracter.com



Anexos: tabla completa de evals · coste de desarrollo

Tabla completa: validador × brief, antes y después del tuning 1

Validador	antes b2	antes b3	antes b4	después b2	después b3	después b4	después b5
brief_schema	✓	✓	✓	✓	✓	✓	✗
forbidden_words_scene	✓	✓	—	✓	✓	✓	—
forbidden_words_chapter	✓	✓	—	✓	✓	✓	—
chapter_length	✓	✓	—	✓	✓	✓	—
exact_names	✓	✓	—	✓	✓	✓	—
brief_coverage	✗	✗	—	✓	✓	✓	—
prose_repetition	✓	✓	—	✓	✓	✓	—
judge_chapter	✓	✓	—	✓	✓	✗	—
judge_novel	✓	✓	—	✓	✓	✗	—
lean_chronology	✓	✓	—	✓	✓	✓	—
visual_check	✓	✓	—	✓	✓	✓	—
coste USD	1,12	1,24	0,25	0,97	1,02	1,93	—

b2 infantil · b3 inyección · b4 trampas temporales · b5 contradictorio (rechazado en el brief). Fuente: [evals/results.md](#) y [evals/results/{before,after}/*.json](#). Detalle: [presentacion/anexo-evals-tabla.pdf](#).

Cómo reduciríamos el coste de desarrollo del harness

27,03 USD

gasto real de modelo en todo el desarrollo

53 %

en intentos bloqueados de 10 capítulos (14,33 USD)

«El coste de desarrollo no está en los tokens; está en el retrabajo.»

- 1 Spec del producto antes que harness genérico**
el harness de partida (~20.000 líneas, novela larga genérica) hubo que adaptarlo casi entero; empezar por spec + esqueleto de punta a punta
- 2 Iterar con novelas cortas**
los fallos ya salían en 1–3 capítulos (0,4–0,9 USD); la de 10 solo al final
- 3 Lo determinista, antes de escribir**
fechas, cronología y lugares validados en el plan: segundos y céntimos, frente a una novela entera
- 4 Tests sin modelo**
FakeModelClient y respuestas grabadas; en vivo solo un smoke pequeño
- 5 Modelo según la tarea**
potente para diseño y specs; barato para tests, docs y regenerar tipos; presupuesto por agente
- 6 Contratos primero, pocos ficheros compartidos**
K1–K5 evitaron retrabajo; rutas y dependencias compartidas costaron fusiones
- 7 Verificación proporcional al riesgo**
TLC y Lean con modelos pequeños; gate estricto solo en el núcleo
- 8 Reutilizar**
Langfuse, FastMCP, Playwright MCP, reportlab; llevar hooks, skills y el runner de evals a MyFactory
- 9 Observabilidad desde el día 1**
coste por llamada y por rol: se optimiza donde está el gasto (writer y editor)